ASSIGNMENT 6 SOLUTION STEP BY STEP SCREENSHOTS

1. **MySQL Database Creation**

```
mysql> USE ecommerce_db;
Database changed
```

2. **Table Creation**

```
mysql> USE ecommerce_db;
Database changed
mysql>
mysql> CREATE TABLE IF NOT EXISTS orders (
    ->     order_id     INT AUTO_INCREMENT PRIMARY KEY,
    ->     customer_id  INT NOT NULL,
    ->     product_name VARCHAR(255) NOT NULL,
    ->     amount       DECIMAL(10, 2) NOT NULL,
    ->     status       ENUM('pending','confirmed','shipped','delivered','cancelled') DEFAULT 'pending
    ->     created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ->     updated_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
    -> );
Query OK, 0 rows affected (0.03 sec)
```

3. **Table Structure Verification**

```
mysql> DESCRIBE orders;
+--------------+-----------------------------------------------------------------------+------+-----+-------------------+-
----------------------------------------+
| Field        | Type                                                                  | Null | Key | Default           | E
                                       |
+--------------+-----------------------------------------------------------------------+------+-----+-------------------+-
----------------------------------------+
| order_id     | int                                                                   | NO   | PRI | NULL              | a
ncrement                               |
| customer_id  | int                                                                   | NO   |     | NULL              |
                                       |
| product_name | varchar(255)                                                          | NO   |     | NULL              |
                                       |
| amount       | decimal(10,2)                                                         | NO   |     | NULL              |
                                       |
| status       | enum('pending','confirmed','shipped','delivered','cancelled')         | YES  |     | pending           |
                                       |
| created_at   | timestamp                                                             | YES  |     | CURRENT_TIMESTAMP | D
T_GENERATED                            |
| updated_at   | timestamp                                                             | YES  |     | CURRENT_TIMESTAMP | D
T_GENERATED on update CURRENT_TIMESTAMP |
+--------------+-----------------------------------------------------------------------+------+-----+-------------------+-
----------------------------------------+
7 rows in set (0.00 sec)
```

4. **CDC Configuration**

```
mysql> SHOW VARIABLES LIKE 'log_bin';
+---------------+-------+
| Variable_name | Value |
+---------------+-------+
| log_bin       | ON    |
+---------------+-------+
1 row in set (0.00 sec)

mysql> SHOW VARIABLES LIKE 'binlog_format';
+---------------+-------+
| Variable_name | Value |
+---------------+-------+
| binlog_format | ROW   |
+---------------+-------+
1 row in set (0.00 sec)

mysql> SHOW VARIABLES LIKE 'server_id';
+---------------+-------+
| Variable_name | Value |
+---------------+-------+
| server_id     | 1     |
+---------------+-------+
1 row in set (0.00 sec)
```

5. **Data Insertion**

```
mysql> USE ecommerce_db;
Database changed
mysql>
mysql> INSERT INTO orders (customer_id, product_name, amount, status) VALUES
    -> (101, 'Laptop Pro 15', 1299.99, 'pending'),
    -> (102, 'Wireless Headphones', 89.99, 'confirmed'),
    -> (103, 'Gaming Chair', 450.00, 'shipped'),
    -> (104, 'USB-C Hub', 45.50, 'delivered'),
    -> (105, 'Mechanical Keyboard', 129.99, 'pending');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

6. **Data Output**

```
mysql> SELECT * FROM orders;
+----------+-------------+---------------------+---------+-----------+---------------------+---------------------
| order_id | customer_id | product_name        | amount  | status    | created_at          | updated_at
+----------+-------------+---------------------+---------+-----------+---------------------+---------------------
|        1 |         101 | Laptop Pro 15       | 1299.99 | pending   | 2026-04-18 18:48:08 | 2026-04-18 18:48
|        2 |         102 | Wireless Headphones |   89.99 | confirmed | 2026-04-18 18:48:08 | 2026-04-18 18:48
|        3 |         103 | Gaming Chair        |  450.00 | shipped   | 2026-04-18 18:48:08 | 2026-04-18 18:48
|        4 |         104 | USB-C Hub           |   45.50 | delivered | 2026-04-18 18:48:08 | 2026-04-18 18:48
|        5 |         105 | Mechanical Keyboard |  129.99 | pending   | 2026-04-18 18:48:08 | 2026-04-18 18:48
+----------+-------------+---------------------+---------+-----------+---------------------+---------------------
5 rows in set (0.00 sec)
```

7. **Update / Delete Operations**

```
mysql> INSERT INTO orders (customer_id, product_name, amount, status) VALUES (301, 'iPhone 16 Pro', 1199.00, 'pend

Query OK, 1 row affected (0.00 sec)

mysql> UPDATE orders SET status = 'confirmed' WHERE order_id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> DELETE FROM orders WHERE order_id = 5;
Query OK, 1 row affected (0.01 sec)
```

```
mysql> INSERT INTO orders (customer_id, product_name, amount, status) VALUES (301, 'iPhone 16 Pro', 1199.00, 'pending');
Query OK, 1 row affected (0.00 sec)

mysql> UPDATE orders SET status = 'confirmed' WHERE order_id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> INSERT INTO orders (customer_id, product_name, amount, status) VALUES (301, 'MacBook Pro', 2499.00, 'pending');
Query OK, 1 row affected (0.01 sec)

mysql> UPDATE orders SET status = 'shipped' WHERE order_id = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> DELETE FROM orders WHERE order_id = 5;
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO orders (customer_id, product_name, amount, status)
    -> VALUES (302, 'Sony PS5', 1500.00, 'pending');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> UPDATE orders SET status = 'delivered' WHERE order_id = 3;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE orders SET amount = 2499.00 WHERE order_id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> INSERT INTO orders (customer_id, product_name, amount, status) VALUES (302, 'Sony PS5', 1500.00, 'pending');
Query OK, 1 row affected (0.01 sec)

mysql> UPDATE orders SET status = 'delivered' WHERE order_id = 3;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 1  Changed: 0  Warnings: 0
```

8. Library Installation

[1]:
```
import subprocess
subprocess.run(['pip', 'install', 'kafka-python', 'mysql-replication'])
```

[1]:
```
CompletedProcess(args=['pip', 'install', 'kafka-python', 'mysql-replication'], returncode=0)
```

[2]:
```
from kafka import KafkaProducer
from pymysqlreplication import BinLogStreamReader
print("✅ kafka-python ready")
print("✅ mysql-replication ready")
```

```
✅ kafka-python ready
✅ mysql-replication ready
```

[3]:
```
import subprocess
result = subprocess.run(['java', '-version'], capture_output=True, text=True)
print(result.stdout)
print(result.stderr)
```

```
java version "21.0.3" 2024-04-16 LTS
Java(TM) SE Runtime Environment (build 21.0.3+7-LTS-152)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.3+7-LTS-152, mixed mode, sharing)
```

9.Kafta installation

```
Done!
```

[9]:
```
# Install a lightweight Kafka alternative that runs purely in Python
import subprocess
result = subprocess.run(
    ['pip', 'install', 'kafka-python', 'pymysql'],
    stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True
)
print(result.stdout[-500:])
print("Done!")
```

```
Requirement already satisfied: kafka-python in c:\users\hp\documents\python scripts\lib\site-packages (2.3
Requirement already satisfied: pymysql in c:\users\hp\documents\python scripts\lib\site-packages (1.1.2)

Done!
```

10.MySQL Connection in Python

[12]:
```
import pymysql

conn = pymysql.connect(
    host='127.0.0.1',
    user='root',
    password='2463',      # your MySQL root password
    database='ecommerce_db'
)
cursor = conn.cursor()
cursor.execute("SELECT * FROM orders")
rows = cursor.fetchall()
for row in rows:
    print(row)
conn.close()
```

```
(1, 101, 'Laptop Pro 15', Decimal('1299.99'), 'pending', datetime.datetime(2026, 4, 18, 18, 48, 8), datetime.datetime(2026, 4, 18, 18, 48, 8))
(2, 102, 'Wireless Headphones', Decimal('89.99'), 'confirmed', datetime.datetime(2026, 4, 18, 18, 48, 8), datetime.datetime(2026, 4, 18, 18, 48, 8
(3, 103, 'Gaming Chair', Decimal('450.00'), 'shipped', datetime.datetime(2026, 4, 18, 18, 48, 8), datetime.datetime(2026, 4, 18, 18, 48, 8))
(4, 104, 'USB-C Hub', Decimal('45.50'), 'delivered', datetime.datetime(2026, 4, 18, 18, 48, 8), datetime.datetime(2026, 4, 18, 18, 48, 8))
(5, 105, 'Mechanical Keyboard', Decimal('129.99'), 'pending', datetime.datetime(2026, 4, 18, 18, 48, 8), datetime.datetime(2026, 4, 18, 18, 48, 8
```

11. Data Fetch Output

```
(1, 101, 'Laptop Pro 15', Decimal('1299.99'), 'pending', datetime.datetime(2026, 4, 18, 18, 48, 8), datetime.datetime(2026, 4, 18, 18, 48, 8))
(2, 102, 'Wireless Headphones', Decimal('89.99'), 'confirmed', datetime.datetime(2026, 4, 18, 18, 48, 8), datetime.datetime(2026, 4, 18, 18, 48,
(3, 103, 'Gaming Chair', Decimal('450.00'), 'shipped', datetime.datetime(2026, 4, 18, 18, 48, 8), datetime.datetime(2026, 4, 18, 18, 48, 8))
(4, 104, 'USB-C Hub', Decimal('45.50'), 'delivered', datetime.datetime(2026, 4, 18, 18, 48, 8), datetime.datetime(2026, 4, 18, 18, 48, 8))
(5, 105, 'Mechanical Keyboard', Decimal('129.99'), 'pending', datetime.datetime(2026, 4, 18, 18, 48, 8), datetime.datetime(2026, 4, 18, 18, 48, 8
```

12. CDC Pipeline Start

CODE:

```python
import pymysql
import json
import queue
import threading
import time
from datetime import datetime
from decimal import Decimal

# ── Kafka-like in-memory message broker ──
kafka_topic = queue.Queue()

DB_CONFIG = {
    'host': '127.0.0.1',
    'user': 'root',
    'password': '2463',
    'database': 'ecommerce_db'
}

def serialize(obj):
    if isinstance(obj, Decimal): return float(obj)
    if isinstance(obj, datetime): return obj.isoformat()
    return str(obj)
```

```python
# ── CDC PRODUCER: polls MySQL for changes ──
def cdc_producer():
    conn = pymysql.connect(**DB_CONFIG)
    cursor = conn.cursor(pymysql.cursors.DictCursor)
    seen = {}
    print("🔄 CDC Producer started - watching orders table...")
    while True:
        cursor.execute("SELECT * FROM orders")
        rows = cursor.fetchall()
        for row in rows:
            oid = row['order_id']
            if oid not in seen:
                event = {"event_type": "INSERT", "data": row, "timestamp": datetime.now().isoformat()}
                kafka_topic.put(event)
                seen[oid] = row.copy()
                print(f"📥 CDC INSERT → order_id={oid}")
            elif seen[oid]['status'] != row['status'] or seen[oid]['amount'] != row['amount']:
                event = {"event_type": "UPDATE", "data": {"before": seen[oid], "after": row}, "timestamp": datetime.now().isoformat(
                kafka_topic.put(event)
                seen[oid] = row.copy()
                print(f"📥 CDC UPDATE → order_id={oid}")
        time.sleep(3)
```

```python
# ── CONSUMER 1: Activity Log ──
activity_log = []
def consumer_activity_log(event):
    data = event['data'].get('after', event['data']) if event['event_type'] == 'UPDATE' else event['data']
    line = f"[{event['timestamp']}] {event['event_type']:7s} | order={data.get('order_id')} | {data.get('product_name')} | ₹{float(data.get('amount',
    activity_log.append(line)
    print(f"📋 LOG: {line}")

# ── CONSUMER 2: High Value Alert ──
alerts = []
def consumer_high_value(event):
    data = event['data'].get('after', event['data']) if event['event_type'] == 'UPDATE' else event['data']
    if event['event_type'] == 'DELETE': return
    if float(data.get('amount', 0)) >= 1000:
        msg = f"🚨 HIGH VALUE ALERT! order={data.get('order_id')} | {data.get('product_name')} | ₹{float(data.get('amount',0)):,.2f}"
        alerts.append(msg)
        print(msg)

# ── CONSUMER 3: Status Summary ──
status_counts = {}
def consumer_status_summary(event):
    data = event['data'].get('after', event['data']) if event['event_type'] == 'UPDATE' else event['data']
    status = data.get('status', 'unknown')
    status_counts[status] = status_counts.get(status, 0) + 1
    print(f"📊 STATUS SUMMARY: {dict(status_counts)}")
```

```python
# ── CONSUMER 4: Customer Stream ──
customer_data = {}
def consumer_customer_stream(event):
    data = event['data'].get('after', event['data']) if event['event_type'] == 'UPDATE' else event['data']
    cid = str(data.get('customer_id', '?'))
    if cid not in customer_data:
        customer_data[cid] = {'orders': 0, 'revenue': 0.0}
    if event['event_type'] == 'INSERT':
        customer_data[cid]['orders'] += 1
        customer_data[cid]['revenue'] += float(data.get('amount', 0))
    print(f"👤 CUSTOMER {cid}: {customer_data[cid]}")

# ── DISPATCHER: routes events to all consumers ──
def dispatcher():
    consumers = [consumer_activity_log, consumer_high_value, consumer_status_summary, consumer_customer_stream]
    print("✅ All 4 consumers listening on topic 'orders.events'...")
    while True:
        try:
            event = kafka_topic.get(timeout=1)
            for consumer in consumers:
                consumer(event)
            print("─" * 60)
        except queue.Empty:
            continue
```

```python
# ── START ALL THREADS ──
threading.Thread(target=cdc_producer, daemon=True).start()
threading.Thread(target=dispatcher, daemon=True).start()

print("🚀 Pipeline running! Now go to MySQL and run INSERT/UPDATE commands!")
print("Watch the output below update in real time...\n")
time.sleep(15)  # run for 15 seconds to show initial data
print("\n✅ Initial load complete!")
print(f"Activity logs: {len(activity_log)}")
print(f"Alerts fired: {len(alerts)}")
print(f"Status summary: {status_counts}")
print(f"Customer data: {customer_data}")
```

OUTPUT:

```
✅ All 4 consumers listening on topic 'orders.events'...
🚀 Pipeline running! Now go to MySQL and run INSERT/UPDATE commands!
Watch the output below update in real time...

🔄 CDC Producer started - watching orders table...
📥 CDC INSERT → order_id=1
📥 CDC INSERT → order_id=2
📥 CDC INSERT → order_id=3
📥 CDC INSERT → order_id=4
📥 CDC INSERT → order_id=5
📋 LOG: [2026-04-18T19:03:22.424845] INSERT  | order=1 | Laptop Pro 15 | ₹1299.99 | pending
🚨 HIGH VALUE ALERT! order=1 | Laptop Pro 15 | ₹1,299.99
📊 STATUS SUMMARY: {'pending': 1}
👤 CUSTOMER 101: {'orders': 1, 'revenue': 1299.99}
──────────────────────────────────────────────────
📋 LOG: [2026-04-18T19:03:22.424845] INSERT  | order=2 | Wireless Headphones | ₹89.99 | confirmed
📊 STATUS SUMMARY: {'pending': 1, 'confirmed': 1}
👤 CUSTOMER 102: {'orders': 1, 'revenue': 89.99}
──────────────────────────────────────────────────
📋 LOG: [2026-04-18T19:03:22.424845] INSERT  | order=3 | Gaming Chair | ₹450.00 | shipped
📊 STATUS SUMMARY: {'pending': 1, 'confirmed': 1, 'shipped': 1}
👤 CUSTOMER 103: {'orders': 1, 'revenue': 450.0}
──────────────────────────────────────────────────
📋 LOG: [2026-04-18T19:03:22.424845] INSERT  | order=4 | USB-C Hub | ₹45.50 | delivered
📊 STATUS SUMMARY: {'pending': 1, 'confirmed': 1, 'shipped': 1, 'delivered': 1}
👤 CUSTOMER 104: {'orders': 1, 'revenue': 45.5}
──────────────────────────────────────────────────
📋 LOG: [2026-04-18T19:03:22.424845] INSERT  | order=5 | Mechanical Keyboard | ₹129.99 | pending
📊 STATUS SUMMARY: {'pending': 2, 'confirmed': 1, 'shipped': 1, 'delivered': 1}
👤 CUSTOMER 105: {'orders': 1, 'revenue': 129.99}
──────────────────────────────────────────────────


✅ Initial load complete!
Activity logs: 5
Alerts fired: 1
Status summary: {'pending': 2, 'confirmed': 1, 'shipped': 1, 'delivered': 1}
Customer data: {'101': {'orders': 1, 'revenue': 1299.99}, '102': {'orders': 1, 'revenue': 89.99}, '103': {'orders': 1, 'revenue': 450.0}, '104': {'order
s': 1, 'revenue': 45.5}, '105': {'orders': 1, 'revenue': 129.99}}
```

12. Real-Time Logs

```
mysql> INSERT INTO orders (customer_id, product_name, amount, status) VALUES (301, 'iPhone 16 Pro', 1199.00, 'pending
Query OK, 1 row affected (0.00 sec)

mysql> UPDATE orders SET status = 'confirmed' WHERE order_id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> INSERT INTO orders (customer_id, product_name, amount, status) VALUES (301, 'MacBook Pro', 2499.00, 'pending')
Query OK, 1 row affected (0.01 sec)

mysql> UPDATE orders SET status = 'shipped' WHERE order_id = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> DELETE FROM orders WHERE order_id = 5;
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO orders (customer_id, product_name, amount, status)
    -> VALUES (302, 'Sony PS5', 1500.00, 'pending');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> UPDATE orders SET status = 'delivered' WHERE order_id = 3;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE orders SET amount = 2499.00 WHERE order_id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> INSERT INTO orders (customer_id, product_name, amount, status) VALUES (302, 'Sony PS5', 1500.00, 'pending');
Query OK, 1 row affected (0.01 sec)

mysql> UPDATE orders SET status = 'delivered' WHERE order_id = 3;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 1  Changed: 0  Warnings: 0
```

```
🚀 PIPELINE RUNNING! Monitoring for 90 seconds...
Go run INSERT/UPDATE in MySQL now!

🔄 CDC Producer watching orders table...
📋 [2026-04-18T19:21:13.297352] INSERT  | order=1 | Laptop Pro 15 | ₹2499.00 | confirmed
🚨 ALERT! order=1 | Laptop Pro 15 | ₹2,499.00
📊 Status: {'confirmed': 1}
👤 Customer 101: {'orders': 1, 'revenue': 2499.0}
──────────────────────────────────────────────────
📋 [2026-04-18T19:21:13.297352] INSERT  | order=2 | Wireless Headphones | ₹89.99 | shipped
📊 Status: {'confirmed': 1, 'shipped': 1}
👤 Customer 102: {'orders': 1, 'revenue': 89.99}
──────────────────────────────────────────────────
📋 [2026-04-18T19:21:13.297352] INSERT  | order=3 | Gaming Chair | ₹450.00 | delivered
📊 Status: {'confirmed': 1, 'shipped': 1, 'delivered': 1}
👤 Customer 103: {'orders': 1, 'revenue': 450.0}
──────────────────────────────────────────────────
📋 [2026-04-18T19:21:13.297352] INSERT  | order=4 | USB-C Hub | ₹45.50 | delivered
📊 Status: {'confirmed': 1, 'shipped': 1, 'delivered': 2}
👤 Customer 104: {'orders': 1, 'revenue': 45.5}
──────────────────────────────────────────────────
📋 [2026-04-18T19:21:13.297352] INSERT  | order=6 | iPhone 16 Pro | ₹1199.00 | pending
🚨 ALERT! order=6 | iPhone 16 Pro | ₹1,199.00
📊 Status: {'confirmed': 1, 'shipped': 1, 'delivered': 2, 'pending': 1}
👤 Customer 301: {'orders': 1, 'revenue': 1199.0}
──────────────────────────────────────────────────
📋 [2026-04-18T19:21:13.297352] INSERT  | order=7 | MacBook Pro | ₹2499.00 | pending
🚨 ALERT! order=7 | MacBook Pro | ₹2,499.00
📊 Status: {'confirmed': 1, 'shipped': 1, 'delivered': 2, 'pending': 2}
👤 Customer 301: {'orders': 2, 'revenue': 3698.0}
──────────────────────────────────────────────────
📋 [2026-04-18T19:21:13.297352] INSERT  | order=8 | Sony PS5 | ₹1500.00 | pending
🚨 ALERT! order=8 | Sony PS5 | ₹1,500.00
📊 Status: {'confirmed': 1, 'shipped': 1, 'delivered': 2, 'pending': 3}
👤 Customer 302: {'orders': 1, 'revenue': 1500.0}

⏱ 61s left | Logs:8 | Alerts:5 | {'confirmed': 1, 'shipped': 1, 'delivered': 2, 'pending': 4}

⏱ 31s left | Logs:8 | Alerts:5 | {'confirmed': 1, 'shipped': 1, 'delivered': 2, 'pending': 4}

⏱ 1s left | Logs:8 | Alerts:5 | {'confirmed': 1, 'shipped': 1, 'delivered': 2, 'pending': 4}

✅ FINAL RESULTS:
  Total events : 8
  Alerts fired : 5
  Status counts: {'confirmed': 1, 'shipped': 1, 'delivered': 2, 'pending': 4}
  Customers    : ['101', '102', '103', '104', '301', '302']
```

13. Alerts

```
[FINAL] Events:8 | Alerts:5 | Customers:6
   Status: {'confirmed': 1, 'shipped': 1, 'delivered': 2, 'pending': 4}
```

14. Status Summary

```
   [STATUS SUMMARY]
      confirmed    # (1)
      shipped      # (1)
      delivered    ## (2)
      pending      #### (4)
```

15. Customer Analytics

```
[LOG] INSERT | order=1 | Laptop Pro 15          | Rs.  2499.00 | confirmed
   [ALERT] HIGH VALUE! order=1 | Laptop Pro 15 | Rs.2,499.00
   [CUSTOMER] 101: {'orders': 1, 'revenue': 2499.0}
[LOG] INSERT | order=2 | Wireless Headphones    | Rs.    89.99 | shipped
   [CUSTOMER] 102: {'orders': 1, 'revenue': 89.99}
[LOG] INSERT | order=3 | Gaming Chair           | Rs.   450.00 | delivered
   [CUSTOMER] 103: {'orders': 1, 'revenue': 450.0}
[LOG] INSERT | order=4 | USB-C Hub              | Rs.    45.50 | delivered
   [CUSTOMER] 104: {'orders': 1, 'revenue': 45.5}
[LOG] INSERT | order=6 | iPhone 16 Pro          | Rs.  1199.00 | pending
   [ALERT] HIGH VALUE! order=6 | iPhone 16 Pro | Rs.1,199.00
   [CUSTOMER] 301: {'orders': 1, 'revenue': 1199.0}
[LOG] INSERT | order=7 | MacBook Pro            | Rs.  2499.00 | pending
   [ALERT] HIGH VALUE! order=7 | MacBook Pro | Rs.2,499.00
   [CUSTOMER] 301: {'orders': 2, 'revenue': 3698.0}
[LOG] INSERT | order=8 | Sony PS5               | Rs.  1500.00 | pending
   [ALERT] HIGH VALUE! order=8 | Sony PS5 | Rs.1,500.00
   [CUSTOMER] 302: {'orders': 1, 'revenue': 1500.0}
[LOG] INSERT | order=9 | Sony PS5               | Rs.  1500.00 | pending
   [ALERT] HIGH VALUE! order=9 | Sony PS5 | Rs.1,500.00
   [CUSTOMER] 302: {'orders': 2, 'revenue': 3000.0}
```

Final result code:

```
import pymysql
from decimal import Decimal

DB_CONFIG = {'host':'127.0.0.1','user':'root','password':'2463','database':'ecommerce_db'}

conn = pymysql.connect(**DB_CONFIG)
cursor = conn.cursor(pymysql.cursors.DictCursor)
cursor.execute("SELECT * FROM orders")
orders = cursor.fetchall()
conn.close()

activity_log, alerts, status_counts, customer_data = [], [], {}, {}

print("="*65)
print("  REAL-TIME ORDER INTELLIGENCE PIPELINE - RESULTS")
print("="*65)

for row in orders:
    row = dict(row)
    amount = float(row['amount'])
    oid, status, cid = row['order_id'], row['status'], str(row['customer_id'])

    line = f"INSERT | order={oid} | {row['product_name']:<22} | Rs.{amount:>9.2f} | {status}"
    activity_log.append(line)
    print(f"[LOG] {line}")

    if amount >= 1000:
        alert = f"[ALERT] HIGH VALUE! order={oid} | {row['product_name']} | Rs.{amount:,.2f}"
        alerts.append(alert)
        print(f"   {alert}")
```

```
    status_counts[status] = status_counts.get(status, 0) + 1
    if cid not in customer_data: customer_data[cid] = {'orders':0,'revenue':0.0}
    customer_data[cid]['orders'] += 1
    customer_data[cid]['revenue'] += amount
    print(f"   [CUSTOMER] {cid}: {customer_data[cid]}")

print("\n[STATUS SUMMARY]")
for s,c in status_counts.items(): print(f"   {s:<12} {'#'*c} ({c})")

print(f"\n[FINAL] Events:{len(activity_log)} | Alerts:{len(alerts)} | Customers:{len(customer_data)}")
print(f"   Status: {status_counts}")

with open('activity_log.txt','w') as f: f.write('\n'.join(activity_log))
with open('high_value_alerts.txt','w') as f: f.write('\n'.join(alerts))
print("\n[SAVED] activity_log.txt and high_value_alerts.txt")
```

16. Final Results

```
================================================================
  REAL-TIME ORDER INTELLIGENCE PIPELINE - RESULTS
================================================================
[LOG] INSERT | order=1 | Laptop Pro 15          | Rs.  2499.00 | confirmed
   [ALERT] HIGH VALUE! order=1 | Laptop Pro 15 | Rs.2,499.00
   [CUSTOMER] 101: {'orders': 1, 'revenue': 2499.0}
[LOG] INSERT | order=2 | Wireless Headphones    | Rs.    89.99 | shipped
   [CUSTOMER] 102: {'orders': 1, 'revenue': 89.99}
[LOG] INSERT | order=3 | Gaming Chair           | Rs.   450.00 | delivered
   [CUSTOMER] 103: {'orders': 1, 'revenue': 450.0}
[LOG] INSERT | order=4 | USB-C Hub              | Rs.    45.50 | delivered
   [CUSTOMER] 104: {'orders': 1, 'revenue': 45.5}
[LOG] INSERT | order=6 | iPhone 16 Pro          | Rs.  1199.00 | pending
   [ALERT] HIGH VALUE! order=6 | iPhone 16 Pro | Rs.1,199.00
   [CUSTOMER] 301: {'orders': 1, 'revenue': 1199.0}
[LOG] INSERT | order=7 | MacBook Pro            | Rs.  2499.00 | pending
   [ALERT] HIGH VALUE! order=7 | MacBook Pro | Rs.2,499.00
   [CUSTOMER] 301: {'orders': 2, 'revenue': 3698.0}
[LOG] INSERT | order=8 | Sony PS5               | Rs.  1500.00 | pending
   [ALERT] HIGH VALUE! order=8 | Sony PS5 | Rs.1,500.00
   [CUSTOMER] 302: {'orders': 1, 'revenue': 1500.0}
[LOG] INSERT | order=9 | Sony PS5               | Rs.  1500.00 | pending
   [ALERT] HIGH VALUE! order=9 | Sony PS5 | Rs.1,500.00
   [CUSTOMER] 302: {'orders': 2, 'revenue': 3000.0}

[STATUS SUMMARY]
   confirmed    # (1)
   shipped      # (1)
   delivered    ## (2)
   pending      #### (4)

[FINAL] Events:8 | Alerts:5 | Customers:6
   Status: {'confirmed': 1, 'shipped': 1, 'delivered': 2, 'pending': 4}

[SAVED] activity_log.txt and high_value_alerts.txt
```